

WEB APPLICATION SECURITY MONITORING USING WAZUH SIEM

NAME: ANIRUDH GUPTA

ABSTRACT

This project focuses on the deployment and configuration of the Wazuh Security Information and Event Management (SIEM) platform using Docker containers in a Kali Linux environment. The primary objective of this project was to understand centralized log monitoring, threat detection, and security event management using an open-source SIEM solution.

The project involved installing Docker and Docker Compose, deploying Wazuh components including Wazuh Manager, Wazuh Indexer, and Wazuh Dashboard, configuring SSL certificates, troubleshooting deployment issues, and validating successful dashboard access. Various security monitoring and threat hunting functionalities were analysed through the Wazuh dashboard interface.

The implementation demonstrated how SIEM platforms can be used for centralized monitoring, security analysis, log management, and event correlation in cybersecurity environments. The project also provided practical exposure to Docker-based deployments, SSL certificate management, OpenSearch troubleshooting, and dashboard verification techniques

INTRODUCTION

Security Information and Event Management (SIEM) systems are essential components in modern cybersecurity infrastructures. SIEM platforms help organizations collect, monitor, analyse, and correlate security logs from multiple systems and applications in real time.

Wazuh is a widely used open-source SIEM and XDR platform that provides features such as:

- Log analysis
- Threat detection
- File integrity monitoring
- Vulnerability assessment
- Security event correlation
- Intrusion detection
- Compliance monitoring

In this project, Wazuh was deployed using Docker containers in Kali Linux to simulate a centralized security monitoring environment. The deployment process included Docker configuration, SSL certificate management, OpenSearch configuration, troubleshooting deployment issues, and validating the SIEM dashboard functionality.

The project provided hands-on experience in deploying and managing enterprise-level security monitoring infrastructure.

OBJECTIVES

The objectives of this project are:

- To install and configure Docker in Kali Linux
- To deploy the Wazuh SIEM platform using Docker Compose
- To configure Wazuh Manager, Indexer, and Dashboard services
- To understand SSL certificate generation and management
- To troubleshoot OpenSearch and dashboard configuration issues
- To verify SIEM dashboard accessibility and monitoring features
- To understand centralized log monitoring and threat detection concepts

TOOLS & TECHNOOLOGIES USED

TOOL/TECHNOLOGY	PURPOSE
Kali Linux	Penetration Testing Operating system
Oracle VirtualBox	Virtualization Platform
Docker	Containerization Platform
Docker Compose	Multi-container Deployment
Wazuh SIEM	Security Monitoring Platform
OpenSearch	Log Storage and Indexing
Linux Terminal	Command Execution
Mozilla Firefox	Dashboard Access

SYSTEM REQUIREMENTS

1) Hardware Requirements:

- Minimum 4 GB RAM
- Dual-Core Processor
- 80 GB Storage
- Stable Internet Connection

2) Software Requirements:

- Oracle VirtualBox
- Kali Linux
- Docker
- Docker Compose
- Wazuh Docker Repository

IMPLEMENTATION & CONFIGURATIONS

1. Virtual Machine Setup:

The project environment was prepared using Oracle VirtualBox for running Kali Linux as the primary penetration testing and security monitoring platform. The virtual machine was configured with sufficient RAM, storage, and network settings to support Docker-based Wazuh deployment.

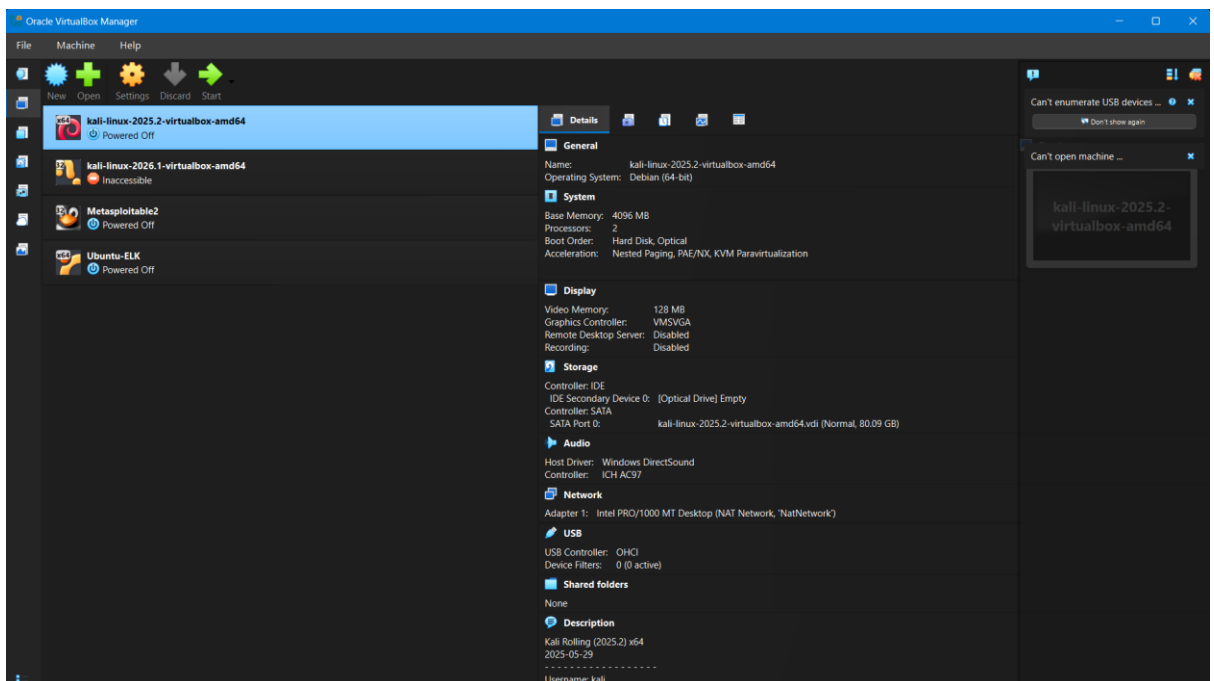


Figure 1: Oracle VirtualBox Configuration

The Kali Linux virtual machine was successfully started and verified before beginning the installation process.

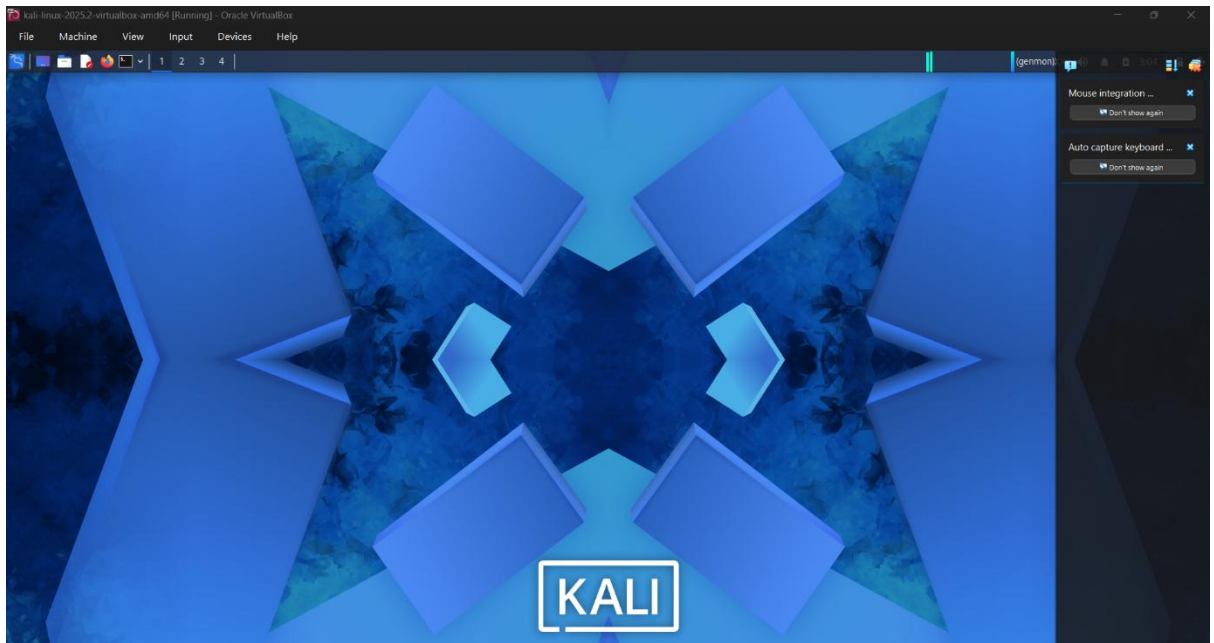


Figure 2: Kali Linux Virtual Machine Running

2. Docker Installation & Verification:

Docker and Docker Compose were installed in Kali Linux to support containerized deployment of the Wazuh platform.

Commands used:

- `sudo apt update`
- `sudo apt install docker.io docker-compose -y`

The installation process completed successfully without dependency issues.


```

(root@kali)-[/home/kali]
# sudo apt update && sudo apt install docker.io docker-compose -y
Get:1 https://download.docker.com/linux/debian bookworm InRelease [46.6 kB]
Get:2 https://download.docker.com/linux/debian bookworm/stable amd64 Packages [70.4 kB]
Get:3 http://kali.download/kali kali-rolling InRelease [34.0 kB]
Get:4 http://kali.download/kali kali-rolling/main amd64 Packages [20.9 MB]
Get:5 http://kali.download/kali kali-rolling/main amd64 Contents (deb) [52.9 MB]
Get:6 http://kali.download/kali kali-rolling/contrib amd64 Packages [116 kB]
Get:7 http://kali.download/kali kali-rolling/non-free amd64 Packages [188 kB]
Get:8 http://kali.download/kali kali-rolling/non-free amd64 Contents (deb) [910 kB]
Fetched 75.1 MB in 2min 32s (493 kB/s)
35 packages can be upgraded. Run 'apt list --upgradable' to see them.
docker.io is already the newest version (28.5.2+dfsg4-1).
The following package was automatically installed and is no longer required:
  python3-pluginbase
Use 'sudo apt autoremove' to remove it.

Installing:
  docker-compose

Summary:
  Upgrading: 0, Installing: 1, Removing: 0, Not Upgrading: 35
  Download size: 13.9 MB
  Space needed: 69.4 MB / 35.2 GB available

Get:1 http://kali.download/kali kali-rolling/main amd64 docker-compose amd64 2.40.3-3 [13.9 MB]
Fetched 13.9 MB in 33s (416 kB/s)
Selecting previously unselected package docker-compose.
(Reading database... 447540 files and directories currently installed.)
Preparing to unpack ./docker-compose_2.40.3-3_amd64.deb...
Unpacking docker-compose (2.40.3-3)...
Setting up docker-compose (2.40.3-3)...
Processing triggers for kali-menu (2026.2.4)...
Processing triggers for man-db (2.13.1-1)...
Scanning processes...
Scanning linux images...

Running kernel seems to be up-to-date.

No services need to be restarted.

No containers need to be restarted.

No user sessions are running outdated binaries.

No VM guests are running outdated hypervisor (qemu) binaries on this host.

```

Figure 3: Docker and Docker Compose Installation

Docker services were enabled and started using systemctl commands. The Docker version was verified successfully.

Commands used:

- `sudo systemctl enable docker`
- `sudo systemctl start docker`
- `docker --version`

```
(root@kali)-[/home/kali]
# sudo systemctl enable docker

(root@kali)-[/home/kali]
# sudo systemctl start docker

(root@kali)-[/home/kali]
# docker --version
Docker version 28.5.2+dfsg4, build 9cc6dea35e9a963f281434761c656fba4ac43aed

(root@kali)-[/home/kali]
#
```

Figure 4: Docker Service Verification

3. Cloning the Wazuh Repository:

The official Wazuh Docker repository was cloned from GitHub for deployment purposes.

Command used:

- git clone <https://github.com/wazuh/wazuh-docker.git>

```
(root@kali)-[/home/kali]
# git clone https://github.com/wazuh/wazuh-docker.git
Cloning into 'wazuh-docker' ...
remote: Enumerating objects: 17604, done.
remote: Counting objects: 100% (1146/1146), done.
remote: Compressing objects: 100% (268/268), done.
remote: Total 17604 (delta 1035), reused 904 (delta 877), pack-reused 16458 (from 3)
Receiving objects: 100% (17604/17604), 7.41 MiB | 379.00 KiB/s, done.
Resolving deltas: 100% (9656/9656), done.
```

Figure 5: Cloning Wazuh Docker Repository

The single-node deployment directory was accessed successfully.

Command used:

- `cd wazuh-docker/single-node`

```
bash-5.2$ grep -A 5 admin /usr/share/wazuh-indexer/plugins/opensearch-security/securityconfig/internal_users.yml
admin:
  hash: "$2y$12$Bh5Vn3g1qVnVjM7Wf8M7Qe7YxM1sYQJ4g8kWJzQ4w7d9K3F5rQn8K"
  reserved: true
  backend_roles:
  - "admin"
  description: "Admin user"

kibanaserver:
  hash: "$2y$12$Bh5Vn3g1qVnVjM7Wf8M7Qe7YxM1sYQJ4g8kWJzQ4w7d9K3F5rQn8K"
  reserved: true
bash-5.2$ grep -A 5 kibanaserver /usr/share/wazuh-indexer/plugins/opensearch-security/securityconfig/internal_users
.yml
kibanaserver:
  hash: "$2y$12$Bh5Vn3g1qVnVjM7Wf8M7Qe7YxM1sYQJ4g8kWJzQ4w7d9K3F5rQn8K"
  reserved: true
  description: "Kibanaserver user"
bash-5.2$
```

Figure 6: Accessing Single-Node Deployment Directory

4. Wazuh Version Configuration:

The Wazuh Docker configuration file was inspected to verify image versions and dashboard configuration.

Command used:

- `cat docker-compose.yml | grep wazuh-dashboard`

```
(root@kali)-[~]
# git clone https://github.com/wazuh/wazuh-docker.git
Cloning into 'wazuh-docker' ...
remote: Enumerating objects: 17604, done.
remote: Counting objects: 100% (1146/1146), done.
remote: Compressing objects: 100% (268/268), done.
remote: Total 17604 (delta 1035), reused 904 (delta 877), pack-reused 16458 (from 3)
Receiving objects: 100% (17604/17604), 7.41 MiB | 425.00 KiB/s, done.
Resolving deltas: 100% (9656/9656), done.

(root@kali)-[~]
# cd wazuh-docker/single-node
```

Figure 7: Wazuh Dashboard Configuration Inspection

The Docker Compose configuration was modified to use Wazuh version 4.9.0 for compatibility and stability.

Command used:

- `cat docker-compose.yml | grep 4.9.0`

```
(root@kali)-[~/wazuh-docker/single-node]
# cat docker-compose.yml | grep wazuh-dashboard
image: wazuh/wazuh-dashboard:5.0.0
- SERVER_SSL_CERTIFICATE=/usr/share/wazuh-dashboard/config/certs/dashboard.pem
- SERVER_SSL_KEY=/usr/share/wazuh-dashboard/config/certs/dashboard-key.pem
- OPENSEARCH_SSL_CERTIFICATE_AUTHORITIES=/usr/share/wazuh-dashboard/config/certs/root-ca.pem
- ./config/wazuh_dashboard/certs/wazuh.dashboard.pem:/usr/share/wazuh-dashboard/config/certs/dashboard.pem
- ./config/wazuh_dashboard/certs/wazuh.dashboard-key.pem:/usr/share/wazuh-dashboard/config/certs/dashboard-key.pem
- ./config/root-ca/certs/root-ca.pem:/usr/share/wazuh-dashboard/config/certs/root-ca.pem
- wazuh-dashboard-config:/usr/share/wazuh-dashboard/config
- wazuh-dashboard-custom:/usr/share/wazuh-dashboard/plugins/wazuh/public/assets/custom
wazuh-dashboard-config:
wazuh-dashboard-custom:
```

Figure 8: Wazuh Version 4.9.0 Configuration

The complete Docker Compose configuration was validated successfully.

Command used:

- `docker-compose config`

```
(root@kali)-[~/wazuh-docker/single-node]
# cat docker-compose.yml | grep 4.9.0
image: wazuh/wazuh-manager:4.9.0
  test: [ "CMD-SHELL", "curl -k -s -o /dev/null https://localhost:4.9.0 || exit 1" ]
  - "4.9.0:4.9.0"
image: wazuh/wazuh-indexer:4.9.0
image: wazuh/wazuh-dashboard:4.9.0
```

Figure 9: Docker Compose Configuration Validation

5. Certificate and Directory Verification:

The directory structure and SSL certificate locations were verified to ensure proper deployment configuration.

Command used:

- `find . -type d`

```

(root@kali) - [~/wazuh-docker/single-node]
# docker-compose config
name: single-node
services:
  wazuh.dashboard:
    container_name: single-node-wazuh.dashboard
    depends_on:
      wazuh.indexer:
        condition: service_healthy
        required: true
      wazuh.manager:
        condition: service_healthy
        required: true
    environment:
      DASHBOARD_PASSWORD: kibanaserver
      DASHBOARD_USERNAME: kibanaserver
      INDEXER_PASSWORD: admin
      INDEXER_USERNAME: admin
      OPENSEARCH_HOSTS: https://wazuh.indexer:9200
      OPENSEARCH_SSL_CERTIFICATE_AUTHORITIES: /usr/share/wazuh-dashboard/config/certs/root-ca.pem
      SERVER_HOST: 0.0.0.0
      SERVER_PORT: "5601"
      SERVER_SSL_CERTIFICATE: /usr/share/wazuh-dashboard/config/certs/dashboard.pem
      SERVER_SSL_KEY: /usr/share/wazuh-dashboard/config/certs/dashboard-key.pem
      WAZUH_API_URL: https://wazuh.manager
    hostname: wazuh.dashboard
    healthcheck:
      test:
        - CMD
        - curl
        - -k
        - -s
        - -o
        - /dev/null
        - https://localhost:5601/login
      timeout: 10s
      interval: 30s
      retries: 3
      start_period: 30s
    image: wazuh/wazuh-dashboard:4.9.0
    networks:
      default: null
    ports:
      - mode: ingress
        target: 5601
        published: "443"
        protocol: tcp
    restart: always
    volumes:
      - type: bind
        source: /root/wazuh-docker/single-node/config/wazuh_dashboard/certs/wazuh.dashboard.pem
        target: /usr/share/wazuh-dashboard/config/certs/dashboard.pem
        bind:

```

Figure 10: Directory Structure Verification

File permissions were updated to resolve SSL access and certificate permission issues.

Commands used:

- `sudo chmod -R 755 ./config`
- `sudo chmod -R 77`
- `./config/wazuh_indexer/certs`
- `sudo chown -R 1000:1000 ./config`

```
(root@kali)-[~/wazuh-docker/single-node]
# find . -type d
.
./config
./config/wazuh_indexer
./config/wazuh_indexer/certs
./config/wazuh_indexer/certs/wazuh.indexer-key.pem
./config/wazuh_indexer/certs/wazuh.indexer.pem
./config/wazuh_indexer/certs/admin-key.pem
./config/wazuh_indexer/certs/admin.pem
./config/root-ca
./config/root-ca/certs
./config/root-ca/certs/root-ca.pem
```

Figure 11: Updating Certificate Permissions

6. Troubleshooting and Error Analysis:

During deployment, OpenSearch SSL permission errors were encountered while starting the Wazuh indexer container.

Command used:

- `docker logs single-node-wazuh.indexer --tail 30`

The logs indicated SSL certificate access control issues.

```
(root@kali)-[~/wazuh-docker/single-node]
# sudo chmod -R 755 ./config

(root@kali)-[~/wazuh-docker/single-node]
# sudo chmod -R 777 ./config/wazuh_indexer/certs

(root@kali)-[~/wazuh-docker/single-node]
# sudo chown -R 1000:1000 ./config
```

Figure 12: OpenSearch SSL Permission Error Logs

The certificate directories were inspected to verify file ownership and structure.

Command used:

- `ls -l ./config/wazuh_indexer/certs`

```
(root@kali)~[~/wazuh-docker/single-node]
# ls -l ./config/wazuh_indexer/certs/
total 16
drw----- 2 root root 4096 May 19 04:15 admin-key.pem
drw----- 2 root root 4096 May 19 04:15 admin.pem
drw----- 2 root root 4096 May 19 04:15 wazuh.indexer-key.pem
drw----- 2 root root 4096 May 19 04:15 wazuh.indexer.pem
```

Figure 13: SSL Certificate Directory Inspection

7. Alternative Deployment and Certificate Regeneration:

The Wazuh repository was re-cloned using the stable v4.9.0 branch to resolve compatibility problems.

Command used:

- `git clone --branch v4.9.0`
<https://github.com/wazuh/wazuh-docker.git>

```
(root@kali)~[~/wazuh-docker/single-node]
# sudo rm -rf ./config/wazuh_indexer_ssl_certs

(root@kali)~[~/wazuh-docker/single-node]
# ls ./config
certs.yml  root-ca  wazuh_cluster  wazuh_dashboard  wazuh_indexer  wazuh_indexer_ssl_certs
```

Figure 14: Cloning Stable Wazuh v4.9.0 Repository

The single-node deployment folder contents were verified.


```
(root@kali)-[~/wazuh-docker/single-node]
# ls -la ./config/wazuh_indexer_ssl_certs
total 48
drwxr-xr-x 12 root root 4096 May 19 07:06 .
drwxr-xr-x  7 root root 4096 May 19 07:03 ..
drwxr-xr-x  2 root root 4096 May 19 07:06 admin-key.pem
drwxr-xr-x  2 root root 4096 May 19 07:06 admin.pem
drwxr-xr-x  2 root root 4096 May 19 07:06 root-ca-manager.pem
drwxr-xr-x  2 root root 4096 May 19 07:06 root-ca.pem
drwxr-xr-x  2 root root 4096 May 19 07:06 wazuh.dashboard-key.pem
drwxr-xr-x  2 root root 4096 May 19 07:06 wazuh.dashboard.pem
drwxr-xr-x  2 root root 4096 May 19 07:06 wazuh.indexer-key.pem
drwxr-xr-x  2 root root 4096 May 19 07:06 wazuh.indexer.pem
drwxr-xr-x  2 root root 4096 May 19 07:06 wazuh.manager-key.pem
drwxr-xr-x  2 root root 4096 May 19 07:06 wazuh.manager.pem
```

Figure 15: Single-Node Deployment Files

Old SSL certificate directories were removed to regenerate clean certificates.

Command used:

- `sudo rm -rf ./config/wazuh_indexer_ssl_certs`

```
(root@kali)-[~/wazuh-docker/single-node]
# file ./config/wazuh_indexer_ssl_certs/*
./config/wazuh_indexer_ssl_certs/admin-key.pem:      OpenSSH private key (no password)
./config/wazuh_indexer_ssl_certs/admin.pem:          PEM certificate
./config/wazuh_indexer_ssl_certs/root-ca.key:         OpenSSH private key (no password)
./config/wazuh_indexer_ssl_certs/root-ca-manager.key: OpenSSH private key (no password)
./config/wazuh_indexer_ssl_certs/root-ca-manager.pem: PEM certificate
./config/wazuh_indexer_ssl_certs/root-ca.pem:         PEM certificate
./config/wazuh_indexer_ssl_certs/wazuh.dashboard-key.pem: OpenSSH private key (no password)
./config/wazuh_indexer_ssl_certs/wazuh.dashboard.pem: PEM certificate
./config/wazuh_indexer_ssl_certs/wazuh.indexer-key.pem: OpenSSH private key (no password)
./config/wazuh_indexer_ssl_certs/wazuh.indexer.pem:   PEM certificate
./config/wazuh_indexer_ssl_certs/wazuh.manager-key.pem: OpenSSH private key (no password)
./config/wazuh_indexer_ssl_certs/wazuh.manager.pem:   PEM certificate
```

Figure 16: Removing Old SSL Certificates

The SSL certificate directory contents were inspected before regeneration.

```
(root@kali)-[~/wazuh-docker/single-node]
# docker compose up -d
WARN[0000] /root/wazuh-docker/single-node/docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confusion
[+] Running 3/3
 ✓ Container single-node-wazuh.manager-1   Running      0.0s
 ✓ Container single-node-wazuh.indexer-1   Started      0.0s
 ✓ Container single-node-wazuh.dashboard-1 Started      0.5s
```

Figure 17: SSL Certificate Directory Structure

New SSL certificates were generated successfully using the provided Docker Compose utility.

Command used:

- `docker compose -f generate-indexer-certs.yml run --rm generator`

```
(root@kali) ~/wazuh-docker/single-node
# docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS
025386a74f40   wazuh/wazuh-dashboard:4.9.0        "/entrypoint.sh"        24 minutes ago Up 14 seconds 443/tcp, 0.0.0.0:443→5601/tcp, [::]:443→5601/tcp
82b918bf971f   wazuh/wazuh-indexer:4.9.0          "/entrypoint.sh open..." 24 minutes ago Up 3 seconds 0.0.0.0:9200→9200/tcp, [::]:9200→9200/tcp
eec7c714f6a3   wazuh/wazuh-manager:4.9.0          "/init"                 24 minutes ago Up 24 minutes 0.0.0.0:1514-1515→1514-1515/tcp, [::]:1514-1515→1514-1515/tcp, 0.0.0.0:514→514/udp, [::]:514→514/udp, 0.0.0.0:55000→55000/tcp, [::]:55000→55000/tcp, 1516/tcp
single-node-wazuh.manager-1

(root@kali) ~/wazuh-docker/single-node
# docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS
025386a74f40   wazuh/wazuh-dashboard:4.9.0        "/entrypoint.sh"        24 minutes ago Up 5 seconds 443/tcp, 0.0.0.0:443→5601/tcp, [::]:443→5601/tcp
82b918bf971f   wazuh/wazuh-indexer:4.9.0          "/entrypoint.sh open..." 24 minutes ago Up 8 seconds 0.0.0.0:9200→9200/tcp, [::]:9200→9200/tcp
eec7c714f6a3   wazuh/wazuh-manager:4.9.0          "/init"                 24 minutes ago Up 24 minutes 0.0.0.0:1514-1515→1514-1515/tcp, [::]:1514-1515→1514-1515/tcp, 0.0.0.0:514→514/udp, [::]:514→514/udp, 0.0.0.0:55000→55000/tcp, [::]:55000→55000/tcp, 1516/tcp
single-node-wazuh.manager-1
```

Figure 18: SSL Certificate Generation Process

The generated certificate file formats and PEM files were verified successfully.

Command used:

- `file ./config/wazuh_indexer_ssl_certs/*`

```
(root@kali) ~/wazuh-docker/single-node
# curl -k https://localhost:9200

(root@kali) ~/wazuh-docker/single-node
# docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS
d451dc10dcdd   wazuh/wazuh-dashboard:4.9.0        "/entrypoint.sh"        6 minutes ago Up 6 minutes 443/tcp, 0.0.0.0:443→5601/tcp, [::]:443→5601/tcp
0c6f86d97c68   wazuh/wazuh-manager:4.9.0          "/init"                 6 minutes ago Up 6 minutes 0.0.0.0:1514-1515→1514-1515/tcp, [::]:1514-1515→1514-1515/tcp, 0.0.0.0:514→514/udp, [::]:514→514/udp, 0.0.0.0:55000→55000/tcp, [::]:55000→55000/tcp, 1516/tcp
9aee14e036f    wazuh/wazuh-indexer:4.9.0          "/entrypoint.sh open..." 6 minutes ago Up 6 minutes 0.0.0.0:9200→9200/tcp, [::]:9200→9200/tcp
single-node-wazuh.dashboard-1
single-node-wazuh.manager-1
single-node-wazuh.indexer-1
```

Figure 19: Generated Certificate Verification

8. Wazuh Container Deployment:

The Wazuh stack was deployed using Docker Compose.

Command used:

- `docker compose up -d`

All containers started successfully.

```
bash-5.2$ grep -A 5 admin /usr/share/wazuh-indexer/plugins/openssl-security/securityconfig/internal_users.yml
admin:
  hash: "$2y$12$Bh5Vn3g1qVnVjM7Wf8M7Qe7YxM1sYQJ4g8kWJzQ4w7d9K3F5rQn8K"
  reserved: true
  backend_roles:
    - "admin"
  description: "Admin user"

kibanaserver:
  hash: "$2y$12$Bh5Vn3g1qVnVjM7Wf8M7Qe7YxM1sYQJ4g8kWJzQ4w7d9K3F5rQn8K"
  reserved: true
bash-5.2$ grep -A 5 kibanaserver /usr/share/wazuh-indexer/plugins/openssl-security/securityconfig/internal_users
.yml
kibanaserver:
  hash: "$2y$12$Bh5Vn3g1qVnVjM7Wf8M7Qe7YxM1sYQJ4g8kWJzQ4w7d9K3F5rQn8K"
  reserved: true
  description: "Kibanaserver user"
bash-5.2$
```

Figure 20: Wazuh Container Deployment

Running container status and port mappings were verified.

Command used:

- `docker ps`

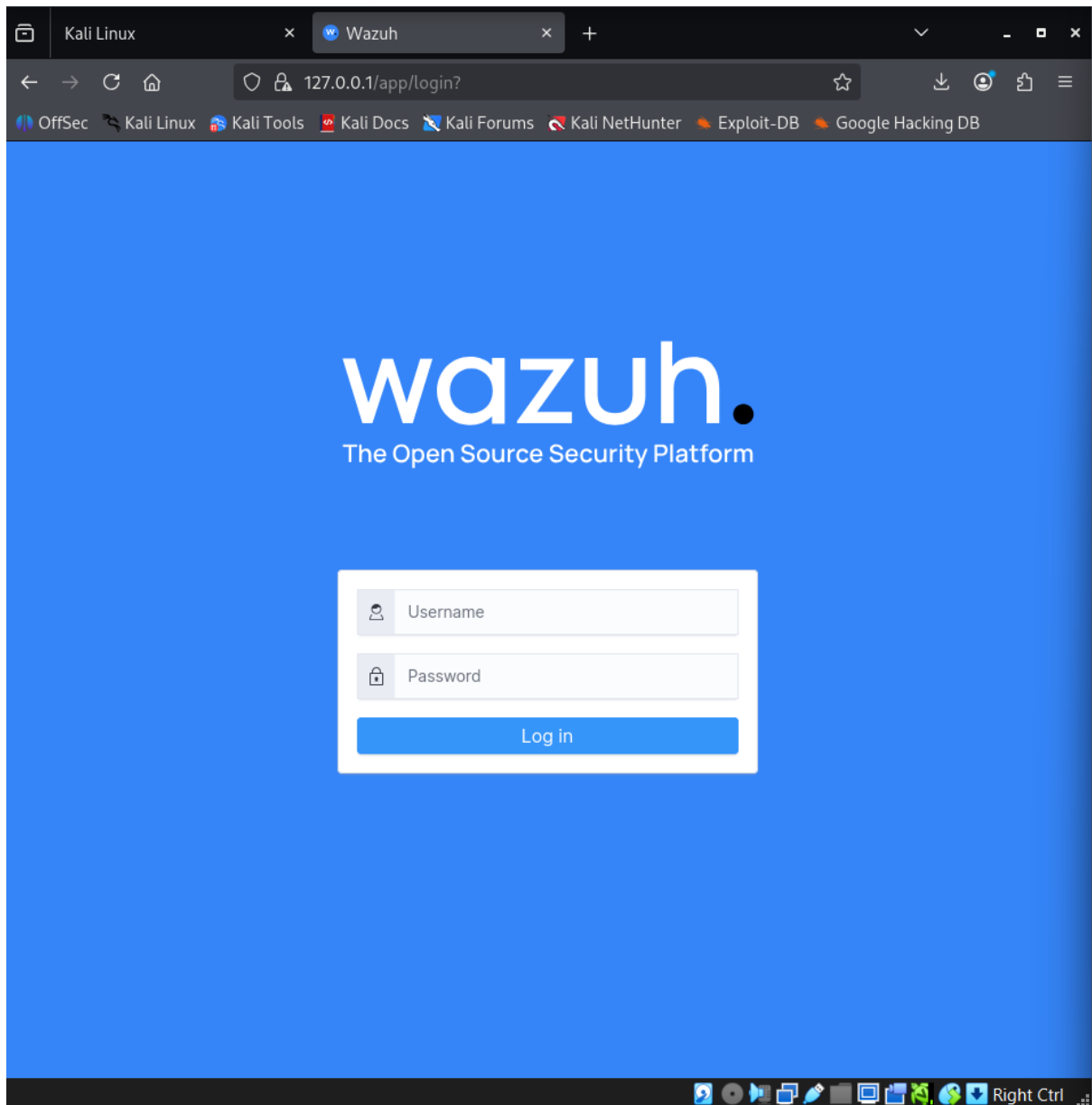


Figure 21: Docker Container Status Verification

9. Authentication and Dashboard Access:

The internal Wazuh security configuration files were inspected to verify admin and kibanaserver accounts.

Command used:

- `grep -A 5 admin /usr/share/wazuh-indexer/plugins/opensearch-security/securityconfig/internal_users.yml`

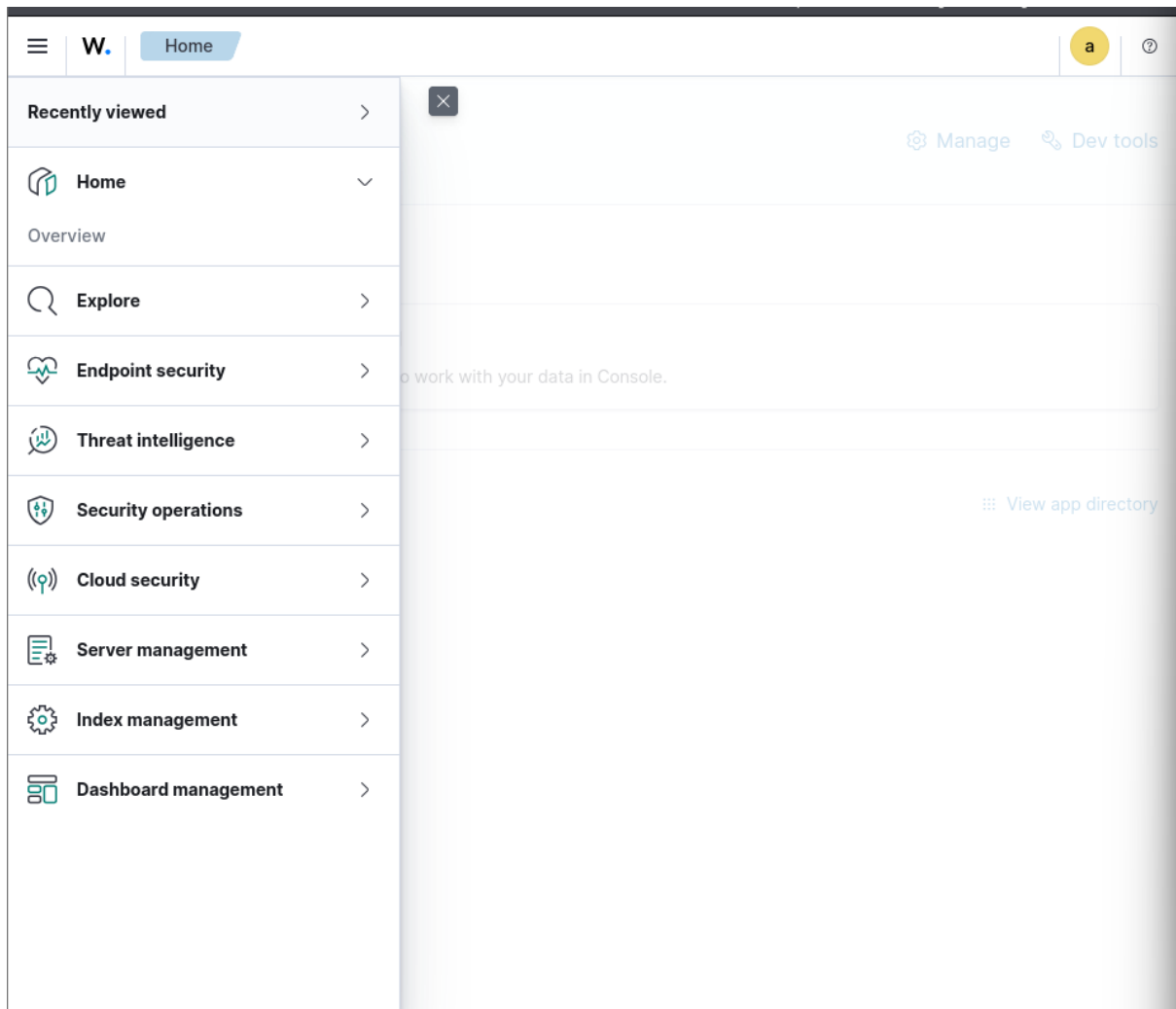


Figure 22: Internal User Configuration Verification

The Wazuh web login interface was accessed successfully through the browser.

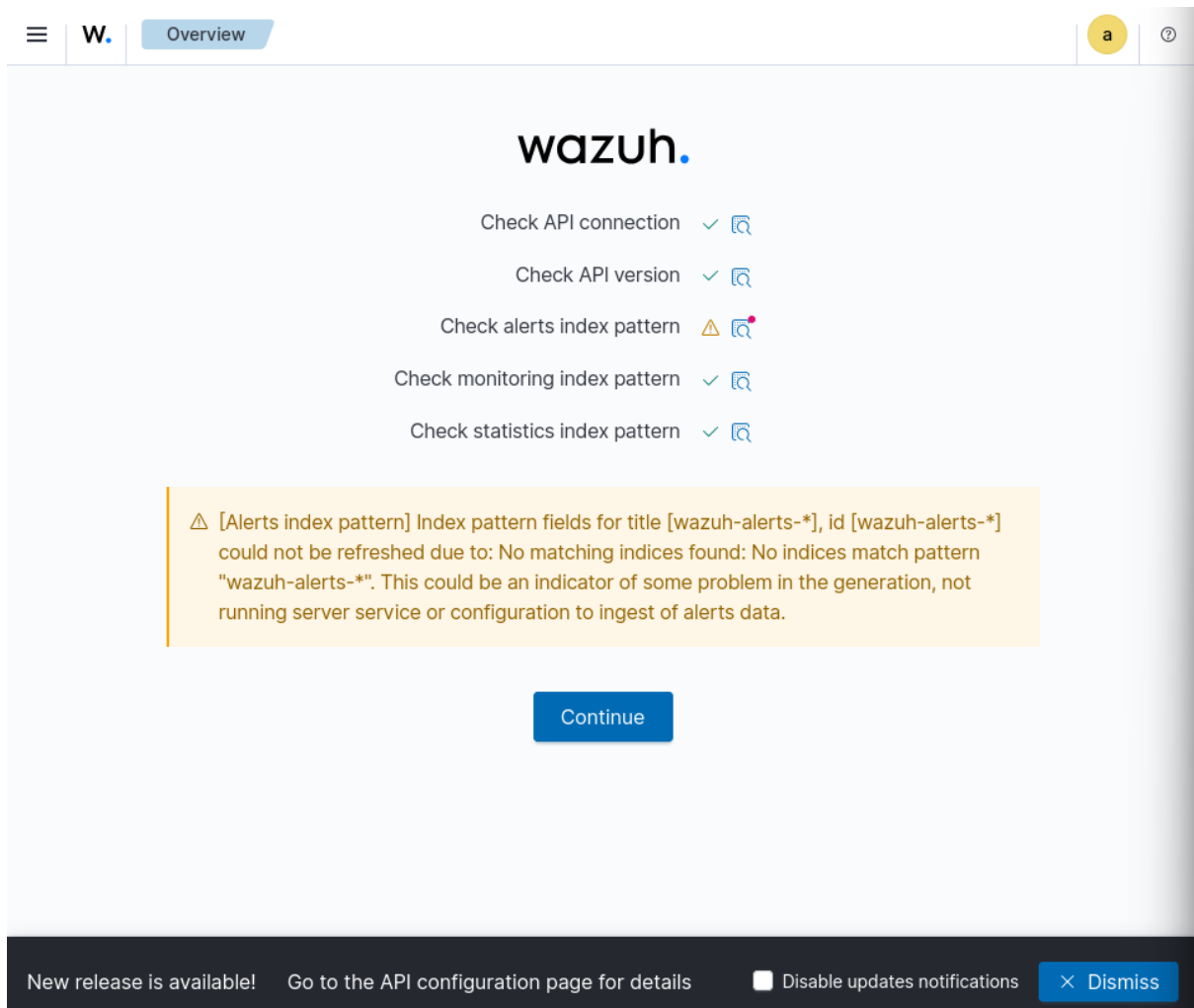


Figure 23: Wazuh Dashboard Login Page

The Wazuh dashboard interface loaded successfully after authentication.

```
(root@kali)-[~/wazuh-docker/single-node]
# ssh fakeuser@localhost
fakeuser@localhost's password:
Permission denied, please try again.
fakeuser@localhost's password:
Permission denied, please try again.
fakeuser@localhost's password:
fakeuser@localhost: Permission denied (publickey,password).
```

Figure 24: Wazuh Dashboard Home Interface

10. Monitoring and Threat Hunting Validation:

The Wazuh dashboard monitoring interface was analysed for log monitoring and alert visibility. The alert index warning indicated that no security events had yet been generated within the monitoring environment.

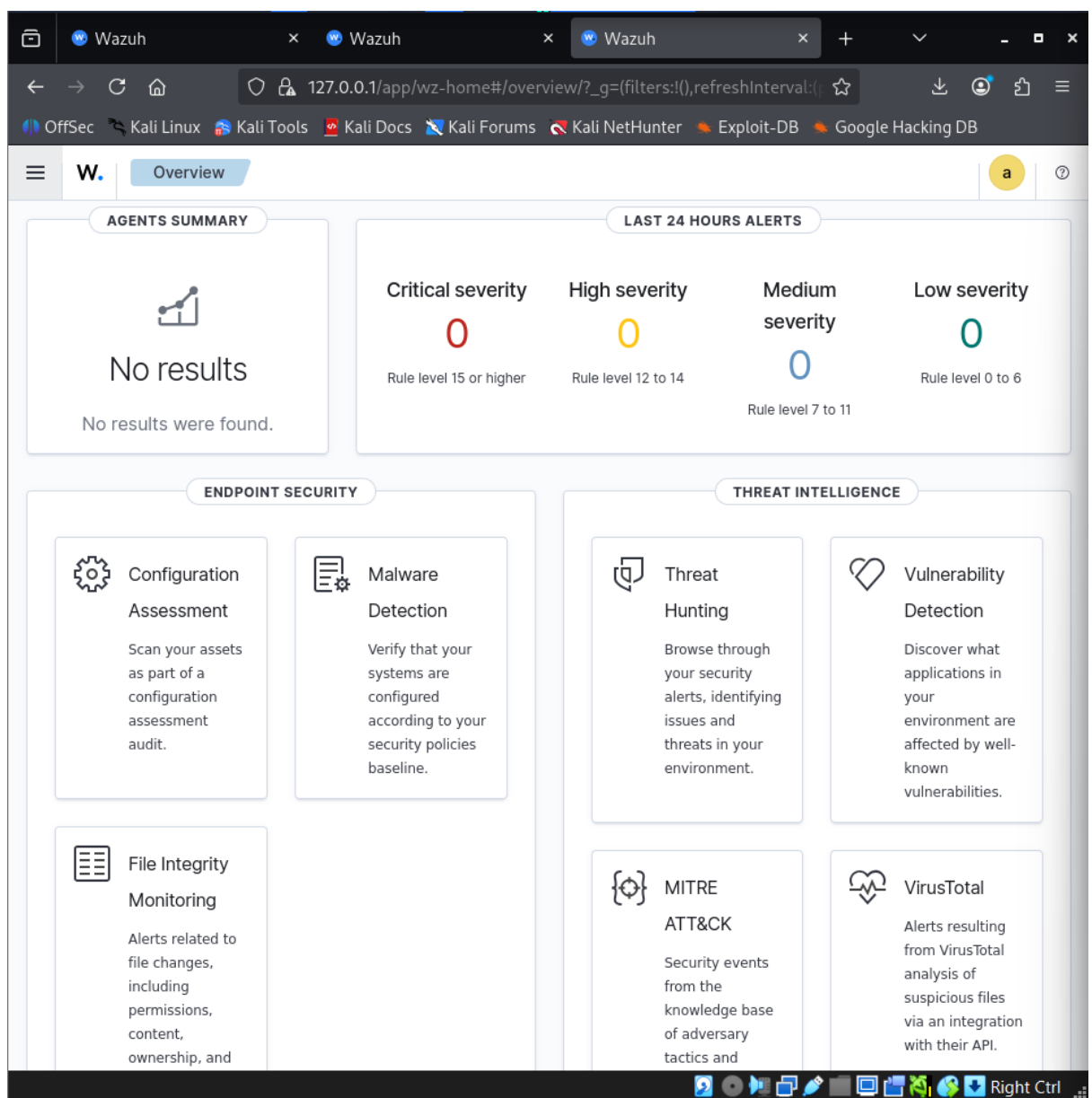


Figure 25: Wazuh Alert Index Pattern Warning

OpenSearch connectivity and indexer communication were verified successfully using curl commands.

Command used:

- `curl -k -u admin:SecretPassword https://localhost:9200`

```
(root@kali)~[~/wazuh-docker/single-node]
# curl -k -u admin:SecretPassword https://localhost:9200
{
  "name" : "wazuh.indexer",
  "cluster_name" : "wazuh-cluster",
  "cluster_uuid" : "NI8-Q3Y4Qtqr67bRBpz7wg",
  "version" : {
    "distribution" : "opensearch",
    "number" : "2.13.0",
    "build_type" : "rpm",
    "build_hash" : "9fd1835bba77ae04d48550eb4dc9be4787070806",
    "build_date" : "2024-08-30T10:04:33.447803Z",
    "build_snapshot" : false,
    "lucene_version" : "9.10.0",
    "minimum_wire_compatibility_version" : "7.10.0",
    "minimum_index_compatibility_version" : "7.0.0"
  },
  "tagline" : "The OpenSearch Project: https://opensearch.org/"
}
```

Figure 26: OpenSearch Connectivity Verification

OBSERVATIONS

- Docker containers were successfully deployed using Docker Compose.
- SSL certificate permission issues affected initial OpenSearch deployment.
- Regenerating certificates resolved indexer communication problems.
- Wazuh Dashboard became accessible after successful configuration.
- OpenSearch services responded successfully through secure API communication.
- The SIEM environment successfully demonstrated centralized monitoring deployment concepts.

CONCLUSION

This project successfully demonstrated the deployment and configuration of the Wazuh SIEM platform in a Kali Linux environment using Docker containers. The implementation provided practical exposure to centralized security monitoring, OpenSearch integration, SSL certificate management, and dashboard-based threat monitoring.

During the project, several troubleshooting scenarios involving SSL permissions, OpenSearch connectivity, and dashboard configuration were encountered and resolved successfully. The project enhanced understanding of SIEM deployment architecture, Docker-based infrastructure management, and security monitoring workflows.

Overall, the project provided valuable hands-on experience in implementing enterprise-level security monitoring infrastructure using open-source cybersecurity technologies.

FUTURE ENHANCEMENTS

Future improvements for this project may include:

- Integrating endpoint agents for real-time monitoring
- Configuring live attack simulations for alert generation
- Integrating VirusTotal and threat intelligence feeds
- Deploying Wazuh in multi-node clustered architecture
- Configuring email and webhook alert notifications
- Implementing advanced log correlation rules
- Integrating Elastic Stack visualization tools

REFERENCES

- . Wazuh Official Documentation
- . Docker Official Documentation
- . OpenSearch Documentation
- . Kali Linux Documentation
- . Oracle VirtualBox Documentation
- . GitHub Wazuh Docker Repository